SnoreGuard: ML-Powered Pillow for Snore Detection and Sleeping Posture Correction

Harkirat Chahal, Dheeraj Kumar, Rohith Bachu, Mohit Gupta,
Anith Chowdary Garikapati, Debanjalee Roy, Harshal Wagh, Pratik Magadwar

***Abstract*— Snoring, often linked to Obstructive Sleep Apnea (OSA), a serious sleep disorder characterized by breathing interruptions, disrupts sleep quality for both the snorer and co-sleepers, affecting overall well-being. The proposed solution leverages an innovative Machine Learning Algorithm for audio analysis integrated into a smart pillow equipped with an air compressor for seamless position adjustment. By addressing the relationship between sleeping position and snoring, this approach seeks to provide a practical means of enhancing sleep quality and promoting a quieter, healthier night's rest for all involved.**

I. INTRODUCTION

Sleep apnea, a prevalent sleep-related breathing issue, involves frequent interruptions in breathing during the night, with obstructive sleep apnea being the most common form. This occurs when the muscles in the upper airway relax during sleep, causing an obstruction that hinders sufficient airflow [1].

Approximately 40% of adult males and 24% of adult females snore regularly [2]. The onset of snoring occurs as throat muscles relax during sleep, constricting the airway and causing vibrations that result in snoring. Snoring is more prevalent when sleeping on the back, and adopting a side-sleeping posture is a natural remedy for snoring [3]. Sleep apnea is often linked to heart disease and metabolic conditions such as diabetes [4]. Numerous studies have demonstrated a connection between sleep apnea and health issues, including type 2 diabetes, strokes, heart attacks, and a shortened lifespan. Snoring disrupts both the quality and duration of sleep for the snorer and their sleeping environment companions. Insufficient and subpar sleep adversely affects memory, cognitive functions, and conflict resolution skills, leading to heightened irritability, a short temper, and feelings of depression [5]. In the United States, insufficient sleep and sleep disorders contribute to annual economic losses of $411 billion, representing 2.28% of the country's gross domestic product (GDP).

The polysomnogram (PSG) is a diagnostic method utilizing electroencephalogram (EEG), electrooculogram (EOG), electromyogram (EMG), electrocardiogram (ECG), pulse oximetry, as well as airflow and respiratory effort to identify the underlying causes of sleep disturbances [6]. However, this diagnostic procedure is time-consuming, expensive, and inconvenient. Patients need to be connected to a minimum of 22 electrodes for several nights, with specialized equipment measuring 11 channels of sleep signals in a laboratory [7]. This presents a barrier for patients who wish to set up and use the device independently at home. Additionally, clinicians must invest significant effort in monitoring and interpreting data to provide an accurate diagnosis. Consequently, the PSG-based approach is costly, complex, and time-consuming, highlighting the need for a more basic, cost-effective, and user-friendly alternative. While some commercially available anti-snoring devices work by advancing the lower jaw to keep the airway open and reduce snoring, these devices are often uncomfortable, impede normal oral function, and require regular cleaning. Thera vent [8], however, employs a nasal strip with micro-valves that generate pressure in the airway during expiration, keeping it open and reducing snoring.

This study introduces a deep learning model for snoring detection utilizing a convolutional neural network (CNN). The model is developed, validated, and tested before being transferred to an Arduino, which functions as the listener module, enabling automatic snoring detection. Additionally, a unique wearable device is designed to emit a vibration warning on the upper arm, prompting the snorer to switch to a side sleeping position. Notably, unlike an alarm sound, the vibration is specific to the snorer, ensuring minimal disturbance to those sleeping nearby.

II. RELATED WORK

The state of the art techniques presented in "A novel deep learning-based approach for sleep apnea detection using single-lead ECG signals" showcase a groundbreaking method for detecting sleep apnea (SA) episodes utilizing electrocardiogram (ECG) data. The authors introduce an enhanced feature extraction technique that concentrates on the identification of S peaks within single-lead ECG data, specifically from the V2 lead, to advance SA detection. These features were employed to educate a Convolutional Neural Network (CNN) model. This approach surpassed existing advanced methods, achieving a classification accuracy of 91.13%, sensitivity of 92.58%, and specificity of 88.75%.

These state of the art techniques underscore the significance of sleep apnea as a common sleep disorder linked to severe health issues such as heart failure and diabetes. They critique the gold standard diagnostic tool, polysomnography, for its complexity and expense, thus underscoring the necessity for a more accessible and cost-effective alternative. The study advocates for ECG as a simpler diagnostic tool for SA detection, given its less cumbersome nature.

In detailing the methodologies, the techniques involve rigorous ECG signal preprocessing, feature extraction, and classifier development. Preprocessing was conducted using a

band-pass filter to eliminate noise and maintain the integrity of the signal. The feature extraction process, especially the amplitude and intervals of R and S peaks, was pivotal and innovative, providing a new perspective on the data. After normalization, these features were input into the SE-ResNext 50 model for signal classification. The experiments leveraged the PhysioNet Apnea–ECG dataset, comprising 70 single-lead ECG records. The data underwent segmentation, labeling, and was enhanced by considering adjacent segments to furnish context for SA detection. A 1D CNN model was meticulously trained and evaluated on this dataset, with experiments to determine the impact of incorporating S peak-related features.

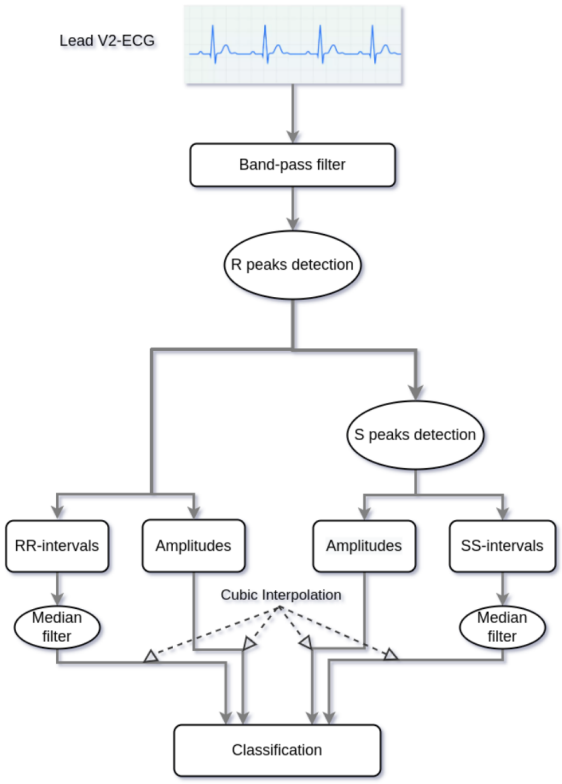

Fig. 1. State-of-the-art model

The results revealed that the inclusion of S peak-related features notably improved the model's performance. This finding implies that changes in the ECG signal related to the S peak bear significant weight in SA classification. The proposed techniques eclipsed several advanced methods, registering impressive accuracy, sensitivity, and specificity metrics. In their conclusion, the researchers deliberate on the potency of the proposed techniques and acknowledge their constraints, such as the limited access to a more extensive dataset of SA patients' ECG data. They propose future work to potentially include the integration of the SA detection module within a mobile application and the exploration of multimodal learning approaches to refine the model's efficacy. They conclude with acknowledgments and references, giving credit to the support and funding that facilitated the research.

These state of the art techniques are pivotal for their potential in presenting a new, more user-friendly, and cost-effective approach to SA detection. The exclusive use of single-lead ECG data and the innovation in focusing on S peaks for classification are substantial contributions to the biomedical signal processing domain.

III. System Design

.

The snore sensing algorithm is intricately designed to perform a series of essential tasks. Initially, the integrated microphone captures relevant snoring sounds during the user's sleep. Subsequently, feature extraction is executed using Mel frequency cepstral coefficients (MFCCs). These coefficients play a pivotal role in compressing and extracting pertinent features from the recorded sound data.

To enable accurate classification of snoring incidents, a Convolutional Neural Network (CNN) is employed as the machine learning model. This model is meticulously trained to distinguish between snoring and non-snoring sounds based on the extracted features. Real-time processing capabilities ensure the immediate identification of snoring incidents as they occur.

Incorporating a remote station for logging purposes, the system facilitates the transmission of data, logging all incidents of snoring for further analysis. Beyond the immediate scope of snore detection, the collected dataset proves to be a valuable resource for the early detection of Sleep Apnea (SA). This potential extends to the integration of alert mechanisms, allowing for timely intervention in cases where Sleep Apnea is suspected.

The project workflow is designed with a focus on user comfort and minimal intervention. The wearable band ensures continuous monitoring, providing a comprehensive dataset for analysis. The system's scalability is noteworthy, as it can be expanded to incorporate additional health monitoring features, thus enhancing its utility in a broader health monitoring context.

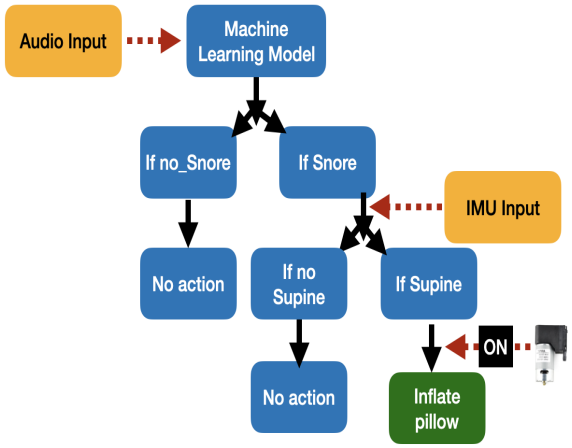

Fig. 2. High level system design

A. High Level Design

In the high-level design illustrated in the figure (2), the system will undergo two phases of machine learning model training for snoring detection and sleeping postures. Upon obtaining these two models, ML Model 1 will determine whether snoring is present. If it detects "snoring," the system will await the prediction of ML Model 2.

There are three possible scenarios, out of which two involve no system response:

Case 1: If ML Model 1 detects no snoring, there will be no response by the system.

Case 2: If ML Model 2 detects a side sleeping position, the system will not respond.

Case 3: If ML Model 1 detects snoring and ML Model 2 detects a supine/prone sleeping position, the system will send a signal to activate the mini air compressor. Subsequently, the air compressor will remain active until the pillow inflates. If the snorer transitions from the supine/prone position to a side sleeping position, the system will reactivate the air compressor to deflate the pillow back to its original shape.

The code developed for this project makes use of a 1D convolutional neural network (CNN) using TensorFlow and Keras. This type of model is particularly suited for sequential or time-series data.

The script begins by importing necessary TensorFlow modules and defining hyperparameters such as the number of epochs, learning rate, batch size, and a determinism flag which, if set to True, avoids non-deterministic operations like shuffling the data. The train dataset is shuffled unless determinism is enforced and both training and validation datasets are batched, with the batch size possibly being a user-supplied argument.

The model architecture is defined as sequential, meaning layers are stacked linearly. It commences with a Reshape layer that transforms the input into a 7x13 matrix from a flat input of length 91. Following the reshaping, the model employs several Conv1D layers with ReLU activations, each succeeded by a MaxPooling1D layer that halves the dimensions, and Dropout layers to prevent overfitting by randomly setting a portion of input units to zero during training.

After the convolutional and pooling layers, the data is flattened into a single vector and passed through a final Dense layer with a softmax activation function, which is used for binary classification as indicated by the output dimension of 2.

The Adam optimizer, a variant of stochastic gradient descent, is configured with a learning rate and hyperparameters beta_1 and beta_2 for momentum calculation. A custom callback, BatchLoggerCallback, is appended for logging purposes and to ensure optional determinism during training.

The model is compiled with a categorical crossentropy loss function, which is appropriate for a classification problem with multiple classes, and accuracy as the metric for evaluation. It is then trained with the defined training dataset, validated against a separate validation dataset, and trained for a specified number of epochs. Verbose output during training is set to level 2, which shows progress per epoch without showing the individual steps within each epoch.

Lastly, a flag to disable per-channel quantization is mentioned, which is a technique that can reduce model size and memory footprint at the potential cost of reduced accuracy. This flag is not applied within the code snippet, so it remains a configurable option.

The working of the main model architecture starts by setting an input length of 91, which represents the size of the input data. The model then reshapes this input into a 7x13 matrix, preparing it for convolutional layers. The first convolutional layer, Conv1D, applies 8 filters with a kernel size of 3 and uses 'same' padding to ensure the output has the same length as the input. The activation function used is ReLU (Rectified Linear Unit), popular for introducing non-linearity to the model without affecting the receptive fields of the convolution layer.

After the convolutional layer, a MaxPooling1D layer reduces the dimensionality of the data by taking the maximum value over a window of size 2 and a stride of 2, which also helps to prevent overfitting by providing an abstracted form of the features. A Dropout layer is added next, randomly setting 25% of the input units to zero during training, which helps in preventing overfitting.

The pattern of Conv1D, MaxPooling1D, and Dropout layers repeats to further abstract and reduce the data dimensionality, with the second Conv1D layer containing 16 filters. Following the convolutional and pooling layers, the Flatten layer transforms the multidimensional data into a 1D array, making it suitable for input into the dense layers.

The final layer is a Dense layer with 2 output units and a '*softmax*' activation function, indicating that the model is intended for a binary classification task. The '*softmax*' activation ensures the output values are in the range (0, 1) and sum to 1, making them interpretable as probabilities.

Finally, "*the model.summary()*" function is called to print a summary of the model's architecture, displaying the configuration of each layer, including the shape transformation that the data undergoes as it progresses through the network.

The Arduino sketch for the implementation and operation of the compressor uses the Edge Impulse Ingestion SDK to perform audio processing for snore detection. It integrates a machine learning model with sensor data to infer snoring events.

The program includes headers for the LSM9DS1 IMU sensor, which likely tracks user movement or orientation, and for Pulse-Density Modulation (PDM) to handle audio data. A custom snore detection library is also included, suggesting a pre-trained model is being used for snore classification.

To manage memory usage on constrained devices like Arduino, a macro is set to 0, which might reduce the memory footprint. The number of slices per model window defines how audio data is segmented for processing by the machine learning model.

An inference_t structure is set up to hold audio buffer pointers and manage state between the PDM callback and the main loop. In the main setup function, the serial com-

munication is initiated, and various model settings are printed out. It initializes the PDM with a microphone and sets up the IMU sensor.

The main loop function reads data from the IMU sensor and records audio via PDM. This audio data is then passed to the snore detection model in a continuous inference loop. If snoring is detected and certain conditions from the IMU sensor are met (e.g., a specific orientation like lying on the back), it triggers a relay, which could activate a device to address the snoring.

A PDM data callback function handles audio buffer filling and switching, facilitating continuous audio data collection. Additionally, functions to start and end microphone inference manage resource allocation and de-allocation.

The script also includes a safeguard, throwing an error if the wrong sensor model is used, ensuring the model and the sensor data are compatible.

B. Low Level Design

As the schematics shown in figure (5), the integration of this snore detection system is an amalgamation of various components, each playing a crucial role in its functionality. At the heart of the system is the Arduino Nano 33 BLE Sense, a compact yet powerful board based on the nRF52840 micro-controller. This board is key to the project as it not only processes audio input for snore detection but also controls the overall operation of the system. It's equipped with a built-in microphone, essential for capturing the snoring sounds, and Bluetooth Low Energy capabilities, which could be used for sending notifications or alerts to a smartphone.

Fig. 3. Relay

The detection of snoring is primarily handled by the microphone integrated into the Arduino Nano 33 BLE Sense. This sensitive audio sensor picks up sound waves, specifically snoring sounds, which are then analyzed by the Arduino to detect snoring patterns. Once snoring is detected, the Arduino triggers a 5V relay. This relay acts as a critical interface between the low-power circuit of the Arduino and the high-power circuit needed for the vacuum pump. It ensures that

Fig. 4. Vacuum Pump

the pump is activated only when necessary, thus conserving energy and ensuring safety.

The vacuum pump, powered by a standard 9V battery, is a pivotal component of the system. It is responsible for inflating or deflating the pillow based on the command received from the Arduino through the relay. The inflatable pillow, a specially designed component of the system, interacts directly with the person. Upon receiving signals of detected snoring, the pillow inflates, gently pushing the person to their side, thereby aiming to reduce or stop the snoring.

Additionally, the power circuitry, which comprises all the wiring and components required to distribute power from the 9V battery to the relay and vacuum pump, is designed to ensure safe and effective power supply to these components. The relay serves as the intermediary device, with its activation contingent upon the detection of snoring incidents by the machine learning algorithm embedded in the Arduino Nano 33 BLE Sense.

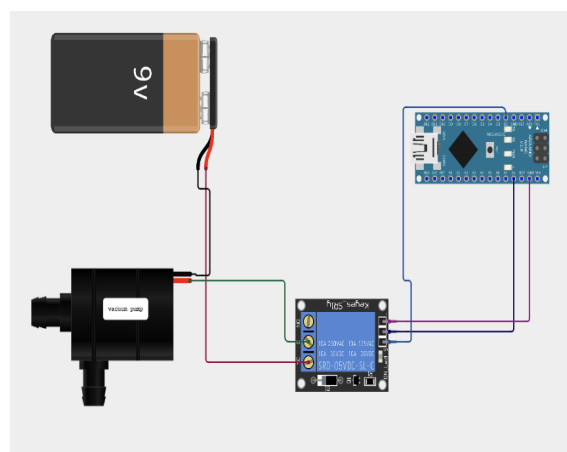

Fig. 5. Circuit Design

When the machine learning algorithm identifies the occurrence of snoring based on the sensory input from the integrated microphone, a signal is sent to the relay from the Arduino Nano 33 BLE Sense. This signal prompts the

relay to switch ON, thereby activating the connected vacuum pump.

The vacuum pump, once activated, initiates the inflation process of the air pillow. This dynamic interaction between the Arduino Nano 33 BLE Sense, the relay, and the vacuum pump creates an automated system that responds to the real-time detection of snoring events.

This careful arrangement of components from the sensing unit (microphone) to the actuation mechanism (pillow and pump) showcases a well-thought-out application of electronics and software in addressing a common issue like snoring. Each component is integral to the system, contributing to its overall goal of improving sleep quality by mitigating snoring. To ensure user convenience and uninterrupted monitoring, the sound detection device is seamlessly integrated into a wearable band, facilitating ease of use and continuous data collection.

IV. Evaluation Approach

The evaluation approach involves the analysis of a public dataset from Kaggle comprising 1000 sound samples. This dataset is structured as follows:

- **Dataset Size:** The dataset consists of 1000 sound samples.
- **Class Distribution:** The dataset is divided into two classes: snoring sounds and non-snoring sounds. Each class contains 500 samples.
- **Sample Duration:** Each sample in the dataset has a duration of one second.
- **Snoring Samples:** There are a total of 500 snoring samples. These samples cover different demographics including children, adult men, and adult women. Out of these, 363 are recordings of snoring sounds with no background noise.
- **Snoring with Background Noise:** 137 snoring samples are recordings of snoring sounds with non-snoring sound backgrounds.
- **Non-Snoring Samples:** The dataset also includes 500 non-snoring samples. These non-snoring samples represent various background sounds that might be present in the vicinity of the snorer.
- **Non-Snoring Sound Categories:** There are ten distinct categories of non-snoring sounds with each category containing 50 samples. These categories encompass a wide range of background sounds including:
  - Baby crying
  - Clock ticking
  - Door opening and closing
  - Total silence
  - Minor sound of a gadget's vibration motor
  - Toilet flushing
  - Siren of an emergency vehicle
  - Rain and thunderstorm
  - Streetcar sounds
  - People talking
  - Background television news
- **Data Preprocessing:** In the data preprocessing phase, we loaded audio recordings from both snoring and no snoring datasets. Using the librosa library, we created spectrograms, which were then normalized for consistency. Finally, as depicted in Figure (6) and Figure (7), these spectrograms were transformed into a 32x32 tensor before being fed into the machine learning model. This preprocessing enables effective pattern recognition between snoring and no snoring events

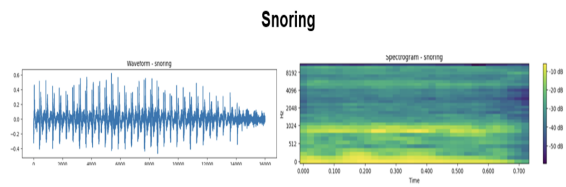

Fig. 6. Waveform: Snoring

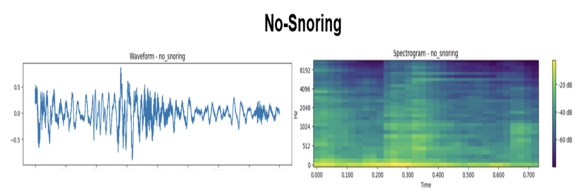

Fig. 7. Waveform: Non-Snoring

In summary, we are utilizing a dataset of 1000 sound samples, categorizing them into snoring and non-snoring classes. The snoring samples include both isolated snoring sounds and snoring sounds with various non-snoring background noises. The non-snoring samples encompass a diverse range of background sounds, each belonging to one of ten distinct categories. This dataset structure facilitates the assessment of sound classification and analysis tasks, particularly for differentiating between snoring and non-snoring sounds in various real-life scenarios.

V. Results

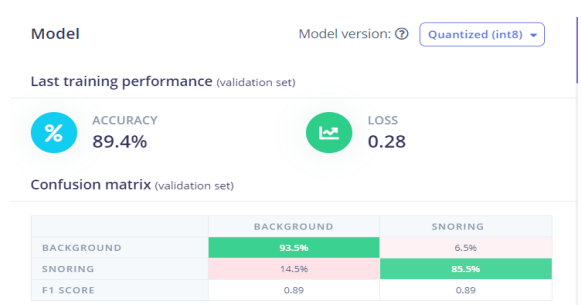

Fig. 8. Model Accuracy

In figure (8), we observe the performance metrics for the machine learning model, specifically highlighting its last training performance on a validation set. The model achieves an accuracy of 89.4%, which indicates a high level

of correct predictions relative to the total predictions made. The reported loss is 0.28, which measures how well the model's predicted outcomes align with the actual values; the lower the loss, the better the model's predictions.

A confusion matrix provides a detailed breakdown of the model's performance. For the 'Background' class, the model accurately identifies 93.5% of the instances, but mistakenly classifies 6.5% of the 'Snoring' instances as 'Background'. Conversely, for the 'Snoring' class, the model correctly identifies 85.5% of instances, while misclassifying 14.5% of them as 'Background'.

The F1 score for both classes is 0.89, which is relatively high, suggesting a balanced precision and recall. Precision is the proportion of true positive predictions in all positive predictions, and recall is the proportion of true positive predictions in all actual positives. An F1 score reaches its best value at 1 (perfect precision and recall) and worst at 0.

The model version is marked as 'Quantized (int8)', indicating that the model weights are quantized to 8-bit integers, a common technique used to reduce the model size and increase the inference speed, typically with a trade-off in precision.

Overall, the metrics depict a model that performs well on the validation set, with strong predictive capabilities, especially considering the quantization, which often affects the model's precision. The high F1 score signifies that the model maintains a good balance between precision and recall, which is crucial for applications where both false positives and false negatives carry significant consequences.

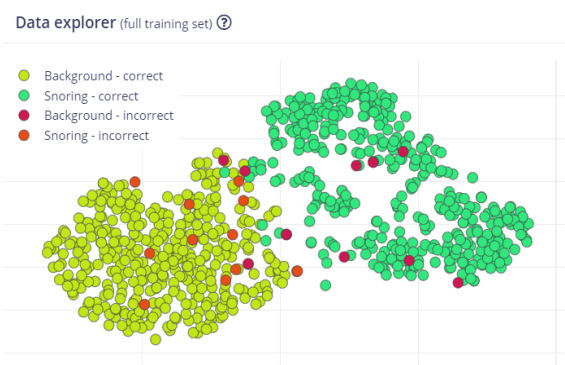

Fig. 9. Scatter plot for performance visualization

Figure (9) appears to be a scatter plot from a data explorer tool, likely used to visualize the performance of a classification model. The plot shows four categories of data points:

Green dots labeled "Background - correct," which suggest these are instances where the model correctly identified background noise. Teal dots labeled "Snoring - correct," indicating these are instances where the model correctly identified snoring sounds. Yellow dots labeled "Background - incorrect," which imply these are instances where the model incorrectly identified something as background noise. Red dots labeled "Snoring - incorrect," showing where the model incorrectly identified something as snoring. The distribution and clustering of the dots could provide insights into the model's performance and how well it can distinguish between snoring and background noise. The clusters may represent the features or dimensions the model uses to make its predictions. This type of visualization is often used in machine learning to evaluate and tune the performance of classification algorithms.

References

[1] Shahrokh Javaheri, Ferran Barbe, Francisco Campos-Rodriguez, Jerome A. Dempsey, Rami Khayat, Sogol Javaheri, Atul Malhotra, Miguel A. Martinez-Garcia, Reena Mehra, Allan I. Pack, et al. *Sleep apnea: types, mechanisms and clinical cardiovascular consequences*.

[2] *Snoring—Overview and Facts*. Available online: http://sleepeducation.org/essentials-in-sleep/snoring/overview-and-facts (accessed on 6 August 2019).

[3] Melone L. *7 Easy Fixes for Snoring*. Available online: https://www.webmd.com/sleep-disorders/features/easy-snoring-remedies1 (accessed on 6 August 2019).

[4] Richard ST Leung and T Douglas Bradley. *Sleep apnea and cardiovascular disease*. American journal of respiratory and critical care medicine, 164(12):2147–2165, 2001.

[5] Breus M.J. *How to Keep Snoring from Hurting Your Relationship*. Available online: https://www.psychologytoday.com/us/blog/sleep-newzzz/201412/how-keep-snoring-hurting-your-relationship (accessed on 6 August 2019).

[6] Jessica Vensel Rundo and Ralph Downey III. *Polysomnography*. Handbook of clinical neurology, 160:381–392, 2019.

[7] Manish Sharma, Shreyansh Agarwal, and U Rajendra Acharya. *Application of an optimal class of antisymmetric wavelet filter banks for obstructive sleep apnea diagnosis using ecg signals*. Computers in biology and medicine, 100:100–113, 2018.

[8] *Theravent*. Available online: https://www.theraventsnoring.com (accessed on 6 August 2019).